

Project Experiments Report

Distributed Social Media Backend

Team: Ruiting Ma, Shuyi Zhang, Sihui Lyu

Introduction

Social media backends must handle massive read volumes and viral traffic spikes without sacrificing latency. This project implements a distributed microservices architecture to navigate the critical trade-offs between caching efficiency, consistency models, and fault tolerance.

We conducted three experiments to isolate and analyze architectural bottlenecks:

- **Caching:** Comparing PostgreSQL-only reads against Redis-materialized timelines to measure database offloading.
- **Consistency:** Evaluating strong vs. eventual consistency for like counts to identify where strict correctness impacts throughput.
- **Resilience:** Simulating replica and cache failures to observe system degradation and recovery.

Codebase: <https://github.com/ruitingm/mini-twitter>

Experiment 1: Redis vs. No Redis

Purpose & Setup

This experiment evaluates whether Redis-backed timeline caching improves read performance compared with direct PostgreSQL reads in a read-heavy social media backend.

The system was seeded with `scripts/seed_test_data_large.sh` and tested with `locustfile.py`. Both configurations ran at 300 users for 5 minutes while CloudWatch tracked RDS, ECS timeline service, ALB, and Redis performance.

Results & Analysis

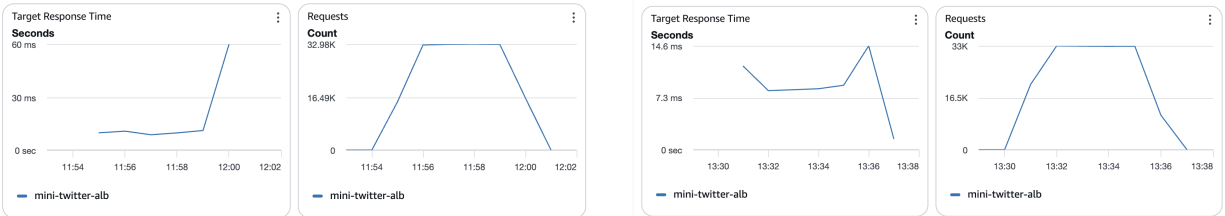
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /v1/timeline/home	28831	0	47	110	370	61.76	35	2037	11185.88	98.8	0
GET	GET /v1/timeline/user/:id	19121	0	45	93	300	56.32	34	1772	11216.99	64.7	0
POST	POST /v1/tweets	9450	0	40	55	130	44.19	31	775	223.52	29.5	0
Aggregated		57402	0	45	95	320	57.05	31	2037	9391.53	193	0

Locust results (300 users, Redis-off)

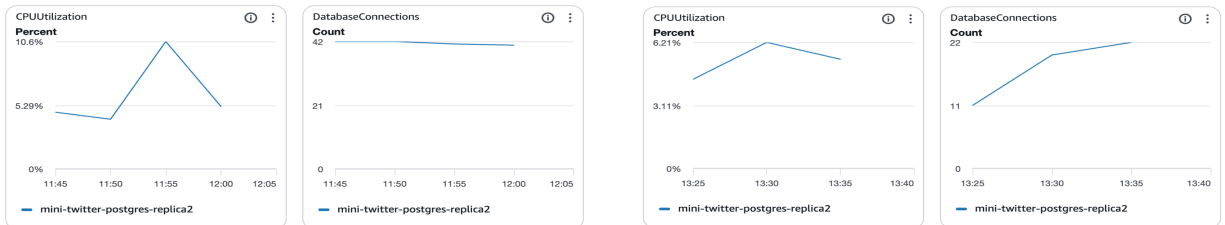
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /v1/timeline/home	28923	0	45	78	130	49.22	33	450	11195.57	95.2	0
GET	GET /v1/timeline/user/:id	19135	0	45	79	140	49.56	33	481	11234.96	65.3	0
POST	POST /v1/tweets	9615	0	42	65	91	43.8	30	430	223.57	32.8	0
Aggregated		57673	0	45	77	130	48.43	30	481	9379.43	193.3	0

Locust results (300 users, Redis-on)

As shown from Locust results, with Redis enabled, aggregated **99th percentile latency dropped (320ms→130ms)** and average latency dropped from ~57ms to ~48ms. Total throughput stayed flat at ~193 RPS. Redis enabled faster reads, especially at the tail, without a big throughput jump.

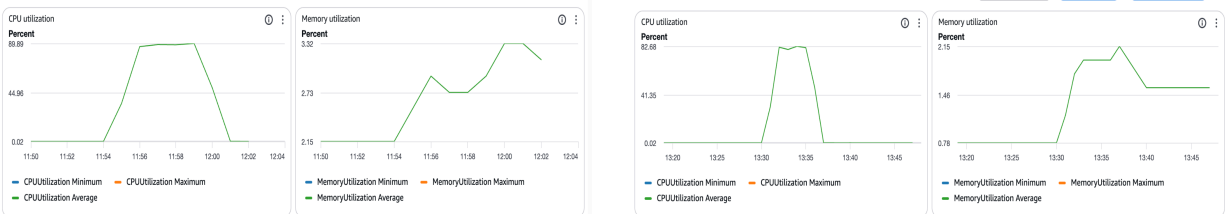


ALB performance (left: Redis-off; right: Redis-on)



RDS read-replica-2 performance (left: Redis-off; right: Redis-on)

The infrastructure metrics support the same conclusion. **ALB target response time fell (~60ms→14.6ms)**. The read replica showed lower **CPU (10.6%→6.2%)** and fewer **connections (42→22)**, reflecting read traffic being offloaded from PostgreSQL.



Timeline ECS cluster performance (left: Redis-off; right: Redis-on)

Timeline service also became slightly cheaper to run. ECS resource utilization dropped (**CPU ~89%→82%, memory ~3.3%→2.1%**), suggesting the service was doing less per-request work when it could read materialized timeline data from Redis instead of relying on database-backed assembly.

Conclusion & Limitations

Overall, experiment 1 produced the expected outcome. Redis-materialized timelines reduced latency across the read path and lowered load on both the database and the timeline service, while overall throughput remained stable.

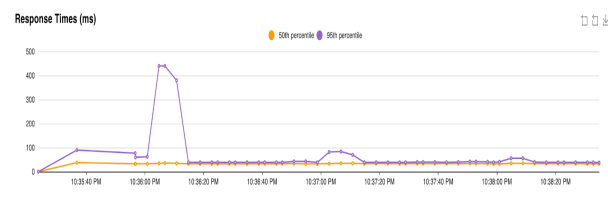
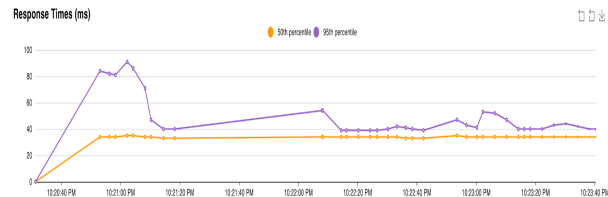
This analysis has limitations since the experiment did not show how the system would behave under longer runs, heavier traffic, or different read/write mixes. In addition, although the Redis-enabled test began with an empty cache, the results did not separate cache warm-up effects from steady-state performance.

Experiment 2: Strong vs. Eventual Consistency

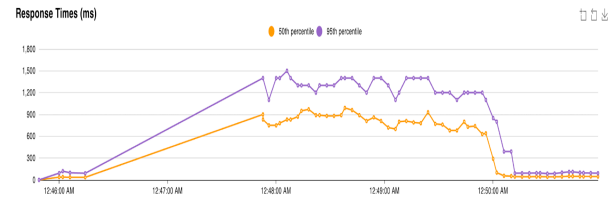
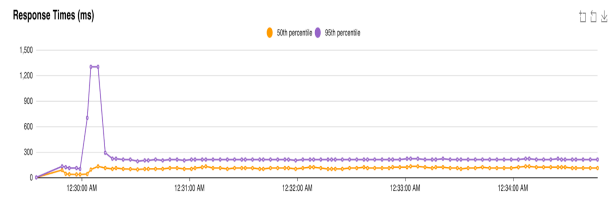
Purpose & Setup

We compare strong consistency (synchronous DB write per like) against eventual consistency (Redis-buffered writes, flushed every 30s). Locust LikeContendingUser has all virtual users target the same hot tweets to maximize contention. We ran experiments at 200 and 500 users to observe how behavior changes with scale.

Results & Analysis



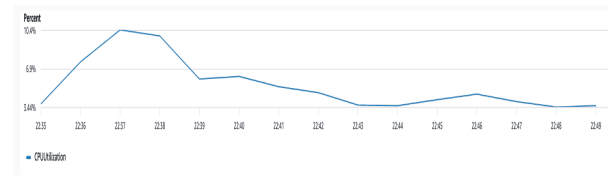
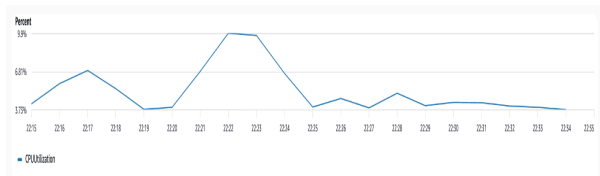
Locust Response Times - 200 users (left: Strong; right: Eventual)



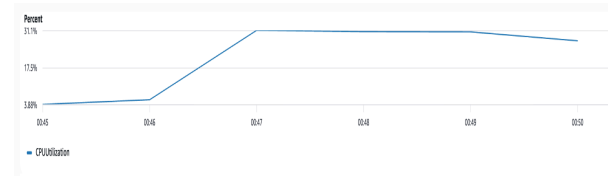
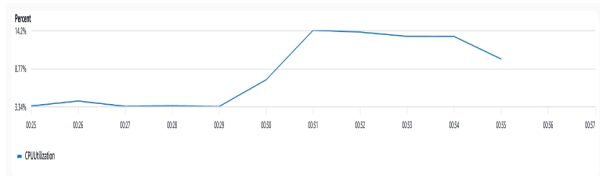
Locust Response Times - 500 users (left: Strong; right: Eventual)

Metric	Strong 200u	Eventual 200u	Strong 500u	Eventual 500u
like p50	34ms	34ms	120ms	830ms
like p95	42ms	48ms	210ms	1300ms
like p99	65ms	280ms	320ms	1600ms
Failures	0	15	0	1734

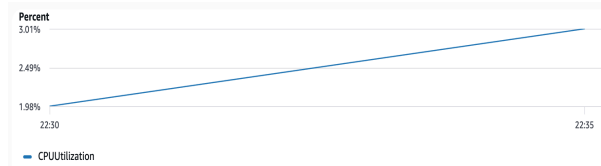
At 200 users, two modes performed similarly. **At 500 users, Eventual mode's p95 was 6x worse and produced 1734 failures.** The cause was the aggregator flush: every 30 seconds ~4500 accumulated writes were released in a single burst, saturating the database. Strong mode distributed writes evenly and remained stable.



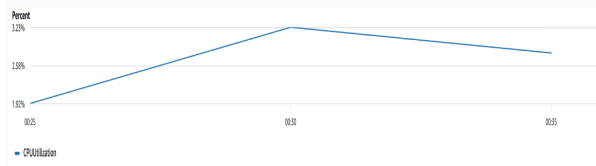
RDS CPU - 200 users (left: Strong; right: Eventual)



RDS CPU - 500 users (left: Strong; right: Eventual)



ElastiCache CPU - 200 users (left: Strong; right: Eventual)



ElastiCache CPU - 500 users (left: Strong; right: Eventual)

Metric	Strong 200u	Eventual 200u	Strong 500u	Eventual 500u
RDS CPU peak	9.9%	10.4%	14.2%	31.1%
ElastiCache CPU peak	2.8%	3%	3.23%	4.13%

Contrary to expectations, **Eventual mode produced higher RDS CPU**, dramatically so at 500 users (31.1% vs. 14.2%). Eventual mode did not eliminate DB writes: each like still performed two synchronous writes (likes + like_count_pending), and the aggregator flush added periodic bulk operations on top. The result was bursty DB load rather than reduced load.

```
=====
Staleness Probe
Tweet : e0e6f587-e3bd-4d6c-99c5-51f6a6a0fcf7
Host : http://mini-twitter-alb-944891444.us-west-2.elb.amazonaws.com
Poll : every 5.0s for 90.0s
=====

Baseline like_count = 4

Time    like_count  delta_from_baseline  Note
-----
T+00s   like_count=4   delta=0              stale (expected >=9, got 4)
T+05s   like_count=4   delta=0              stale (expected >=14, got 4)
T+10s   like_count=4   delta=0              stale (expected >=19, got 4)
T+16s   like_count=4   delta=0              stale (expected >=24, got 4)
T+21s   like_count=4   delta=0              stale (expected >=29, got 4)
T+26s   like_count=4   delta=0              stale (expected >=34, got 4)
T+32s   like_count=38   delta=34             <-- FLUSHED (aggregator ran)
T+37s   like_count=38   delta=34             stale (expected >=44, got 38)
T+43s   like_count=38   delta=34             stale (expected >=49, got 38)
T+48s   like_count=38   delta=34             stale (expected >=54, got 38)
T+53s   like_count=38   delta=34             stale (expected >=59, got 38)
T+59s   like_count=63   delta=59             <-- FLUSHED (aggregator ran)
T+64s   like_count=63   delta=59             stale (expected >=69, got 63)
T+72s   like_count=63   delta=59             stale (expected >=74, got 63)
T+77s   like_count=63   delta=59             stale (expected >=79, got 63)
T+83s   like_count=63   delta=59             stale (expected >=84, got 63)
T+88s   like_count=89   delta=85             <-- FLUSHED (aggregator ran)

=====
Done. Sent 85 likes over 90.0s.
Final like_count = 89 (baseline was 4)
=====
```

Staleness probe terminal output (Eventual mode)

T+00s ~ T+26s	like_count = 4	← 30 likes sent, DB not yet updated
T+32s	like_count = 38	← aggregator flush, count jumps +34
T+37s ~ T+53s	like_count = 38	← another 30 likes accumulate
T+59s	like_count = 63	← second flush

In Eventual mode, users see stale like counts for up to 30 seconds, updating in discrete jumps rather than incrementally. Strong mode has zero staleness.

Conclusion & Limitations

Eventual consistency did not reduce database load as expected. At 500 users, it produced 2x higher RDS CPU, 6x worse p95 latency, and 1734 failures, worse than Strong mode on every metric. The aggregator flush became a bottleneck under high concurrency. The only trade-off unique to Eventual mode was a 30s staleness window with no performance benefit at this scale.

One limitation of this study is that we did not explore tuning the `AGGREGATOR_INTERVAL`; reducing it could potentially decrease both burst size and staleness, introducing a different trade-off profile. In addition, all experiments were conducted on a single `db.t3.micro` instance, so the results may not generalize to setups with horizontally scaled databases. Finally, our eventual consistency implementation still includes a synchronous `like_count_pending` insert for each like as a durability safeguard, which limits the expected write-offloading benefits.

Experiment 3: Failure Resilience

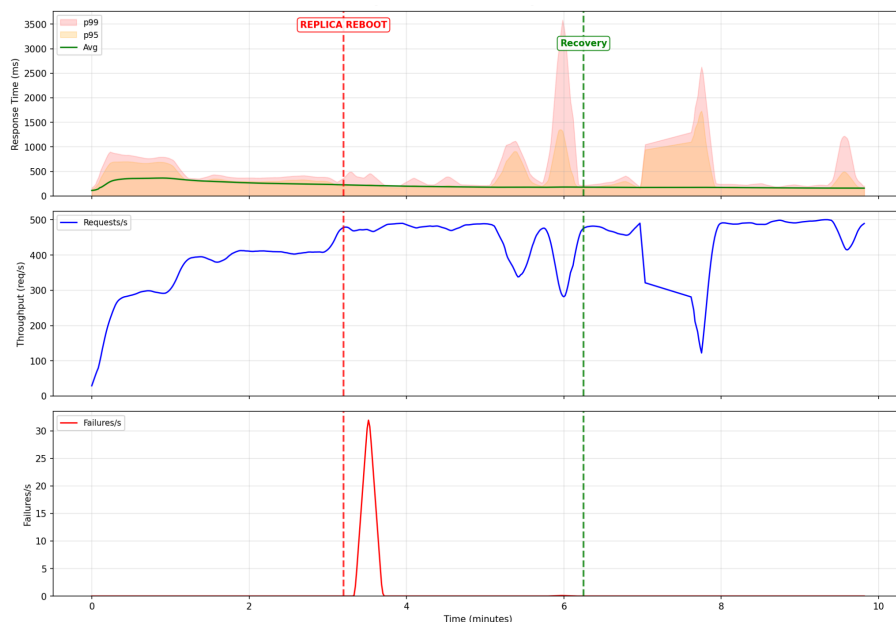
Purpose & Setup

We test whether the system can tolerate a read replica failure and a Redis cache failure without complete service disruption.

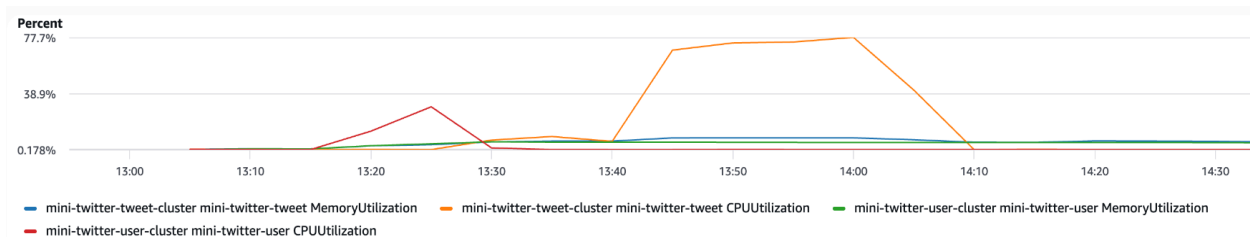
A custom health checker pings each replica every 3s and automatically reroutes reads when one goes down. We ran 200 concurrent users (~500 req/s) across three independent test scenarios: a baseline with all components healthy, a replica failure triggered by `aws rds reboot-db-instance` mid-test, and a Redis cache flush triggered by `FLUSHALL` mid-test.

Results & Analysis

1) Replica Failure



Replica failure



ECS clusters CPU utilization

Window	Avg (ms)	p95	p99	Req/s	Fail/s
Pre-failure	266	302	377	407	0.000
During failure	191	339	609	450	1.983
Post-recovery	169	300	454	458	0.000

The result shows a clear error spike (~30 fails/s) at the moment of reboot, a throughput dip from 450 to 300 req/s, and p99 spikes to 3500ms. The health checker detected the failure within 3-6 seconds and rerouted traffic. Error rate during the failure window was 0.44% (target <1%). Zero failures post-recovery indicates the system self-healed. CloudWatch confirmed the tweet service CPU hit 78%, approaching the auto-scaling threshold.

2) Redis Cache Flush

Window	Avg (ms)	p95	p99	Req/s	Fail/s
Pre-flush	97.7	164	220	502	0.000
Post-flush	96.7	155	216	505	0.000

No measurable impact. **Zero failures, zero latency change.** The PostgreSQL replicas absorbed 100% of reads without Redis at this scale.

Conclusion & Limitations

The system tolerated replica failure gracefully: automatic failover in seconds, 0.44% error rate, full recovery with no manual intervention. Cache loss caused no degradation, the database handled all reads directly at this load level.

The limitation is at 200 users with a 15K-tweet dataset, PostgreSQL's buffer cache serves all queries from memory, making Redis redundant. The expected 3-5x latency increase after cache flush would likely appear at higher concurrency (1000+ users) where database capacity saturates.